

CPE231RC

Algorithms

Lecture10

Special Topic: Genetic Algorithm

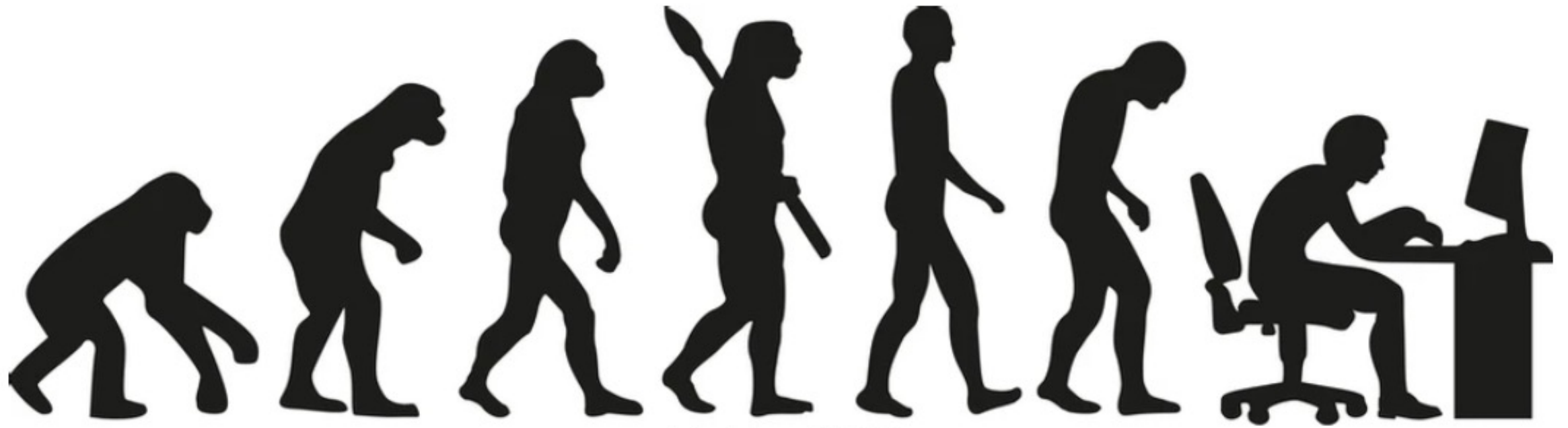
Dr. Prapong Prechaprapranwong



Computer Engineering



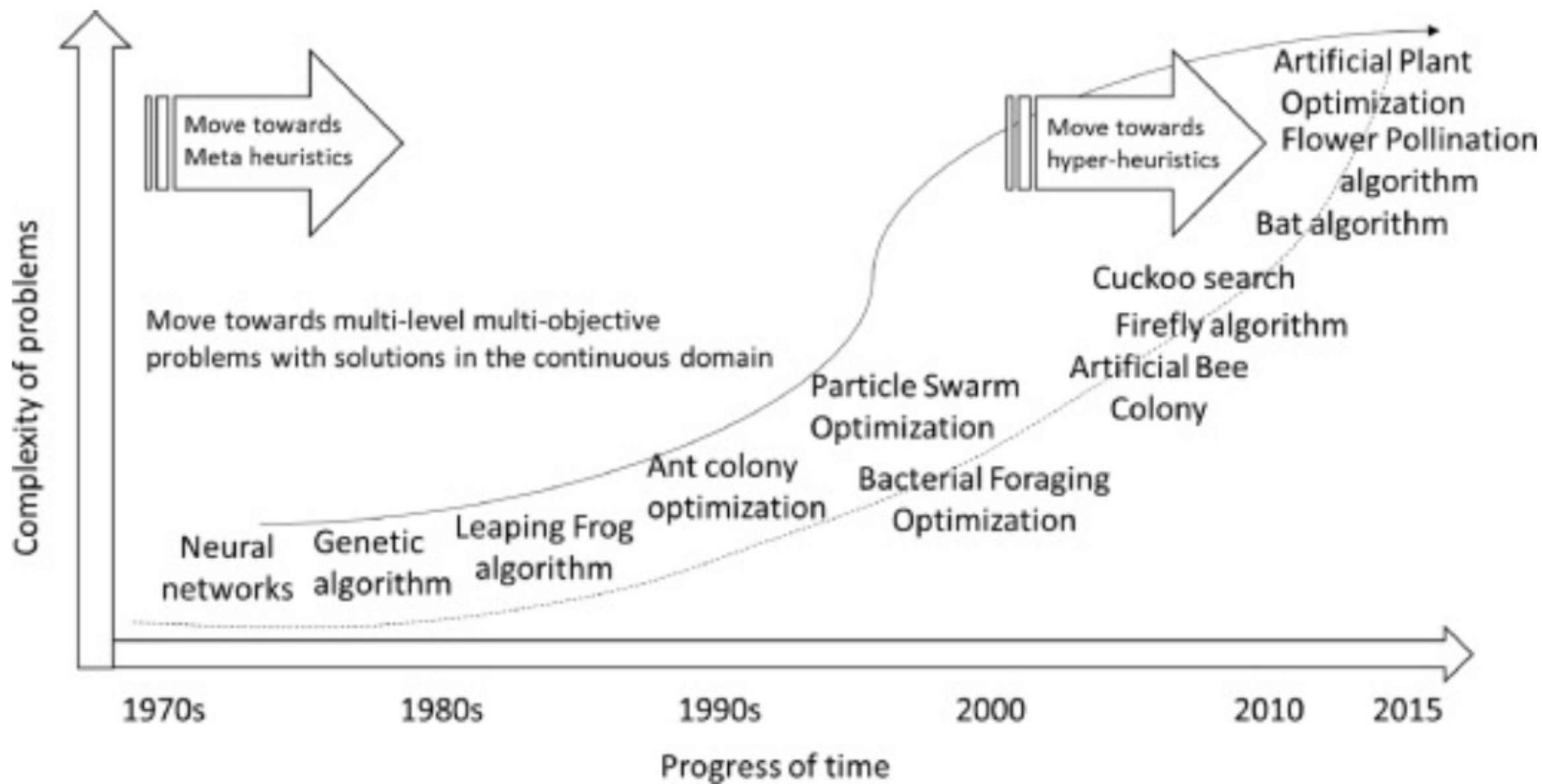
King Mongkut's
University of
Technology
Thonburi



shutterstock.com • 443236687

Overview: Genetic Algorithm

- **Genetic Algorithm (GA)** is one of biology-inspired approach
- GA is based on evolution theory and nature selection
- GA can be seen as an extension of the heuristic search approach
- GA is used to avoid getting stuck in local maximum
- GA creates a space of solutions with many search points
- GA typically applied to discrete optimization





Individual (Chromosomes)



- **Individuals** are search points in solution space that are candidate for a better or (may be) worse solution
- a set of individuals is called a **population**
- In other word, a population is a set of representations of **potential solutions** to the problem
- Succeeding populations constructed in iterated phases of a computation are called **generations**
- **State space** is defined with succeeding generations constructed by a genetic algorithm

Genotype and Phenotype

*“A **genotype** is the genetic makeup of an organism while a **phenotype** is a visible character of an organism.”*

Genotype: S  S alleles W  W

- **Genotype** is an information of the individual in form of binary-codes/number/character/...

→ Encoding



Phenotype: Straight Wings Wrinkled Wings

- **Phenotype** is a represented value of an individual (chromosome)

→ Decoding

Genotype and Phenotype

Example: create a word from numbers

- Genotype

80	114	111	103	114	97	109	109	105	110	103
----	-----	-----	-----	-----	----	-----	-----	-----	-----	-----

P	R	O	G	R	A	M	M	I	N	G
---	---	---	---	---	---	---	---	---	---	---

- Phenotype

“PROGRAMMING”

Genotype and Phenotype

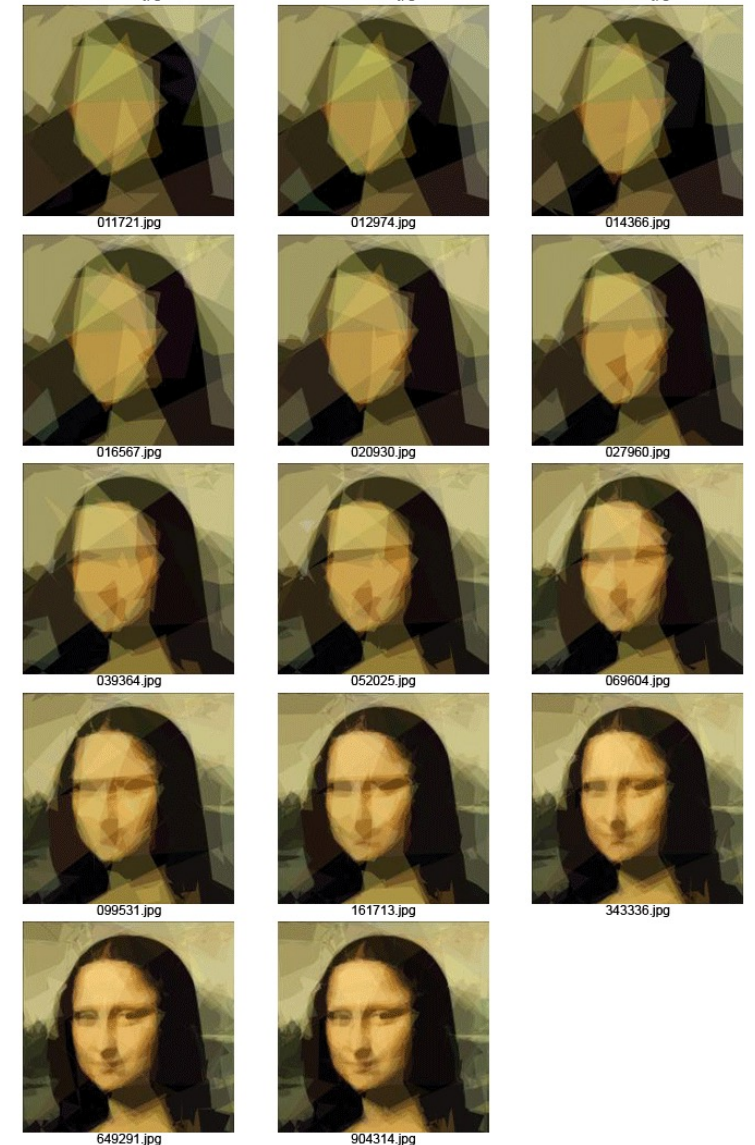
Example: Create Mona Lisa's picture from polygons

- **Genotype**

code of type of polygons

code of color, size and position

- **Phenotype**



Genotype and Phenotype

Examples: if individuals in this problem is defined by the coordinates (X_1, X_2)
the coordinates is turn in to the genotype by a string of genes (binary-codes)

X_2	0111	4	6	7	7	4	1	2	1
	0110	5	6	7	1	4	5	3	3
	0101	1	8	6	2	4	4	7	8
	0100	2	4	3	5	2	5	5	3
	0011	3	7	6	9	1	5	6	2
	0010	5	8	6	6	3	3	7	2
	0001	6	7	6	7	2	5	4	3
	0000	5	6	5	1	1	4	4	2

0000 0001 0010 0011 0100 0101 0110 0111

X_1

let use the 4 bits (genes) represent a position of the coordinates
then X_1 – the first four bits and X_2 – the last four bits

we got a **genotype** of an individual (chromosome) in the form
of 4-bit binary code

e.g., 0001 1000 or 0010 1100, etc.

then we got a **phenotype** by decode a genotype
to a represented value

e.g., from the table

$(X_1, X_2) = (0001, 0010) = 8$

$(X_1, X_2) = (0010, 0011) = 6$

Fitness and Selection

X_2	0111	4	6	7	7	4	1	2	1
	0110	5	6	7	1	4	5	3	3
	0101	1	8	6	2	4	4	7	8
	0100	2	4	3	5	2	5	5	3
	0011	3	7	6	9	1	5	6	2
	0010	5	8	6	6	3	3	7	2
	0001	6	7	6	7	2	5	4	3
	0000	5	6	5	1	1	4	4	2
		0000	0001	0010	0011	0100	0101	0110	0111
		X_1							

- In nature, the strongest will survive. In genetic algorithm use Fitness to indicate that strength
- Fitness is measured in terms of how well the chromosome solves the goal problem
- The most fit chromosome likely will be **selected** to reproduce.
- Fitness can also be subjective
- The fitness function can be subjective (create by your own)

e.g., if we start from the random population of 4 chromosomes and select 2 of the most fit chromosome to reproduce and fitness function is the highest value of phenotype

Population case1:
 (X_1, X_2)
 (0001, 0010) = 8
 (0010, 0011) = 6
 (0011, 0001) = 7
 (0001, 0110) = 6

Arrows from 8 and 7 point to the word "selected".

Population case2:
 (0100, 0001) = 2
 (0011, 0000) = 1
 (0110, 0010) = 7
 (0000, 0001) = 6

Arrows from 7 and 6 point to the word "selected".

Population case3:
 (0101, 0010) = 3
 (0011, 0011) = 9
 (0101, 0110) = 5
 (0101, 0101) = 4

Arrows from 9 and 5 point to the word "selected".

Selection method

Roulette wheel selection

Percent of change to be survived (selected)

(X_1, X_2)

Population case1:

$(0001, 0010) = 8$

$(0010, 0011) = 6$

$(0011, 0001) = 7$

$(0001, 0110) = 6$

$$8+6+7+6 = 27$$

→ 100%

$$(0001, 0010) = 8/27 = 30\%$$

$$(0010, 0011) = 6/27 = 22\%$$

$$(0011, 0001) = 7/27 = 26\%$$

$$(0001, 0110) = 6/27 = 22\%$$

Population case2:

$(0100, 0001) = 2$

$(0011, 0000) = 1$

$(0110, 0010) = 7$

$(0000, 0001) = 6$

$$2+1+7+6 = 16$$

→ 100%

$$(0100, 0001) = 2/16 = 12.5\%$$

$$(0011, 0000) = 1/16 = 6.25\%$$

$$(0110, 0010) = 7/16 = 43.75\%$$

$$(0000, 0001) = 6/16 = 37.5\%$$

Population case3:

$(0101, 0010) = 3$

$(0011, 0011) = 9$

$(0101, 0110) = 5$

$(0101, 0101) = 4$

$$3+9+5+4 = 21$$

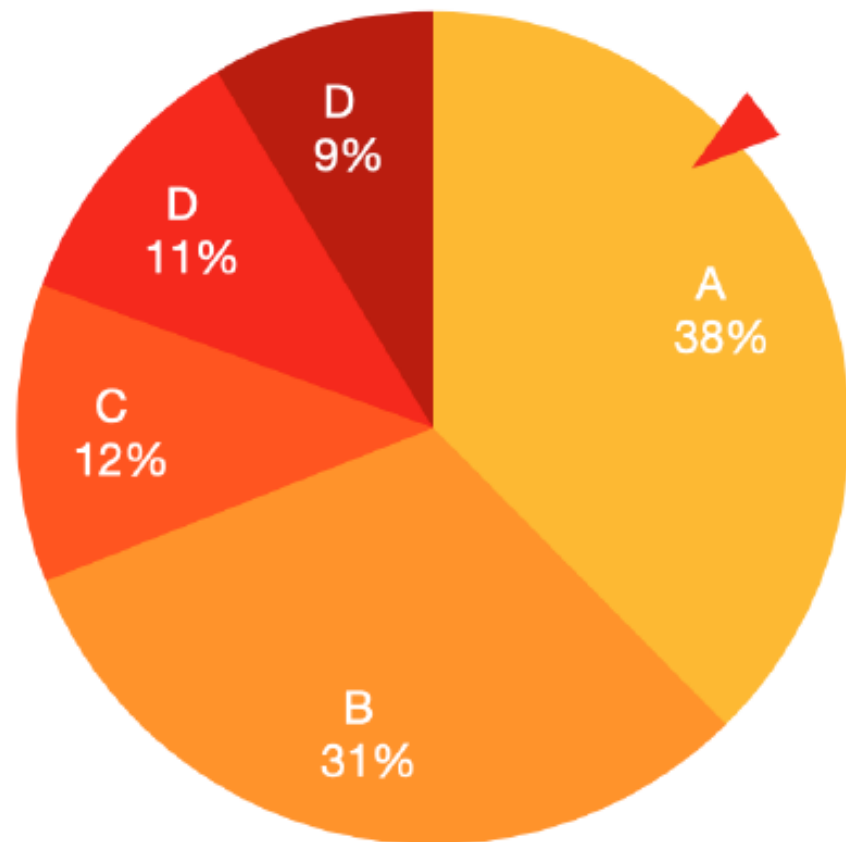
→ 100%

$$(0101, 0010) = 3/21 = 14.3\%$$

$$(0011, 0011) = 9/21 = 42.9\%$$

$$(0101, 0110) = 5/21 = 23.8\%$$

$$(0101, 0101) = 4/21 = 19\%$$



Reproduction

Crossover method

1. Random a crossover point on chromosome string
2. Divide each chromosome into two sub string at the crossover point
3. Re-combine chromosomes by crossover to produce two new chromosomes

e.g.: assume that crossover point is the middle point

case1:

(0001, 0010) = 0001 0010
(0011, 0001) = 0011 0001

New chromosomes (offspring)

0011 0010 = (0011, 0010) = 6
0001 0001 = (0001, 0001) = 7

case2:

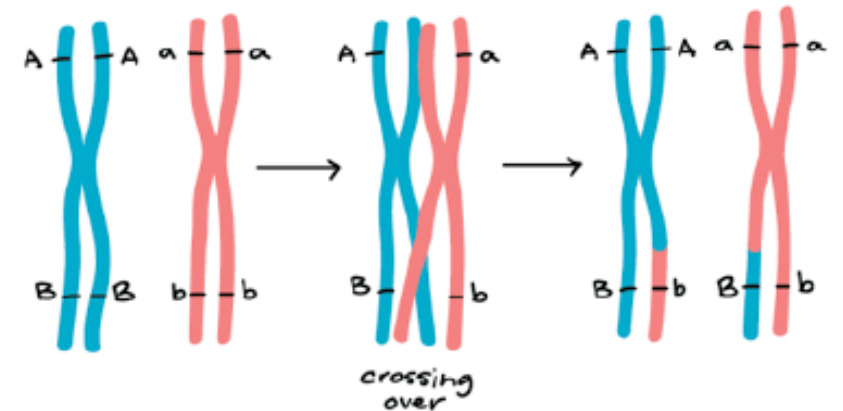
(0110, 0010) = 0110 0010
(0000, 0001) = 0000 0001

0000 0010 = (0000, 0010) = 6
0110 0001 = (0110, 0001) = 4

case3:

(0011, 0011) = 0011 0011
(0101, 0110) = 0101 0110

0101 0011 = (0101, 0011) = 5
0011 0110 = (0011, 0110) = 1



Replace new chromosomes
to population set

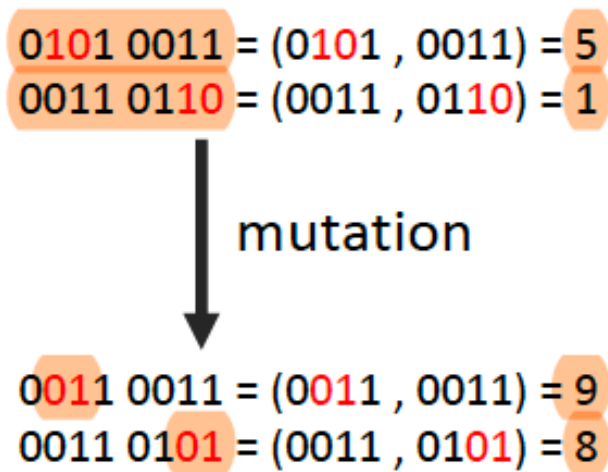
Reproduction

Mutation method

- randomly select some bits (genes) in chromosomes to be flipped
- randomly select some number in chromosomes to be randomly changed

*** Mutation should be not often occurred**

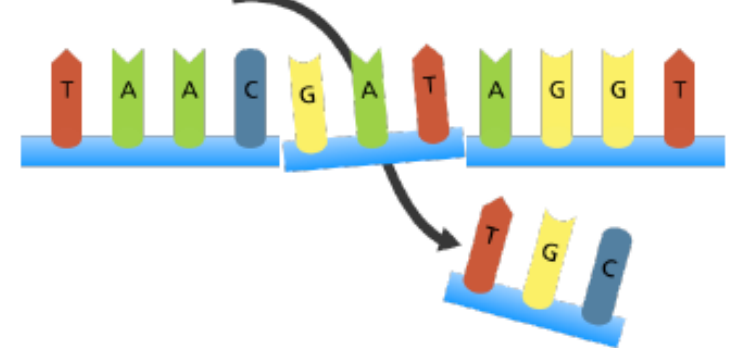
e.g., crossover not improve the fitness of population



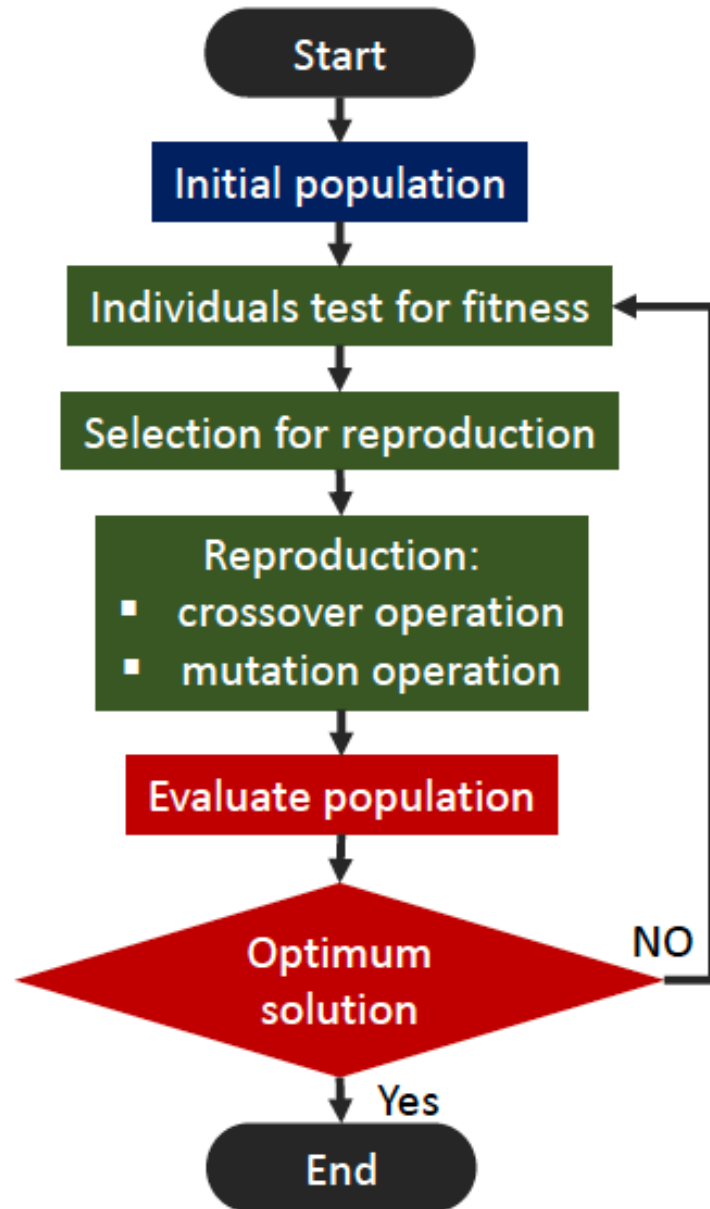
Original sequence



Substitution



Genetic Algorithm: Flowchart



Phase1: Initial population

- Generate a random generation of a fixed number of individuals
- The individuals are initial search points in the solution space

Phase2: Selection for breeding offsprings

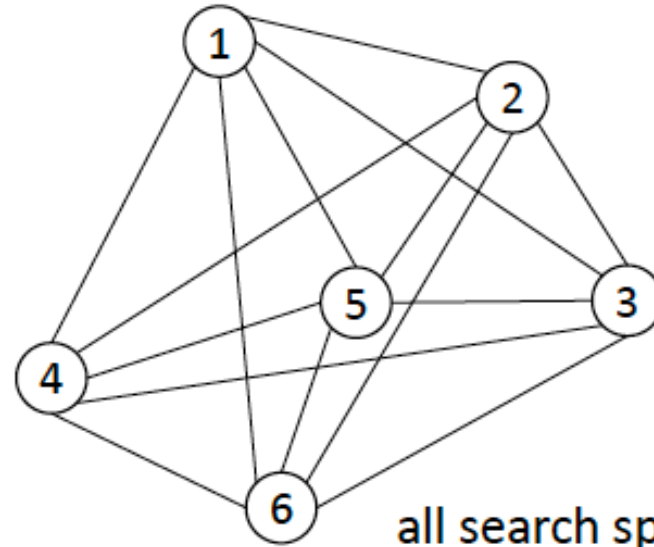
- Test fitness and Select individuals to create a parent population
- The simplest selection method: **roulette wheel selection**
- Reproduction by crossover (re-combination) operator
- Reproduction by mutation operator

Phase3: Evaluation of a population

- Values of the fitness function are computed for individuals of the new offspring population
- Terminate when the optimum solution reached

Example: Traveling salesman problem

Objective: find a complete tour with the minimal path of graph of n cities



all search space = $6! = 720$ paths

Chromosome: order of cities to visit



or



...

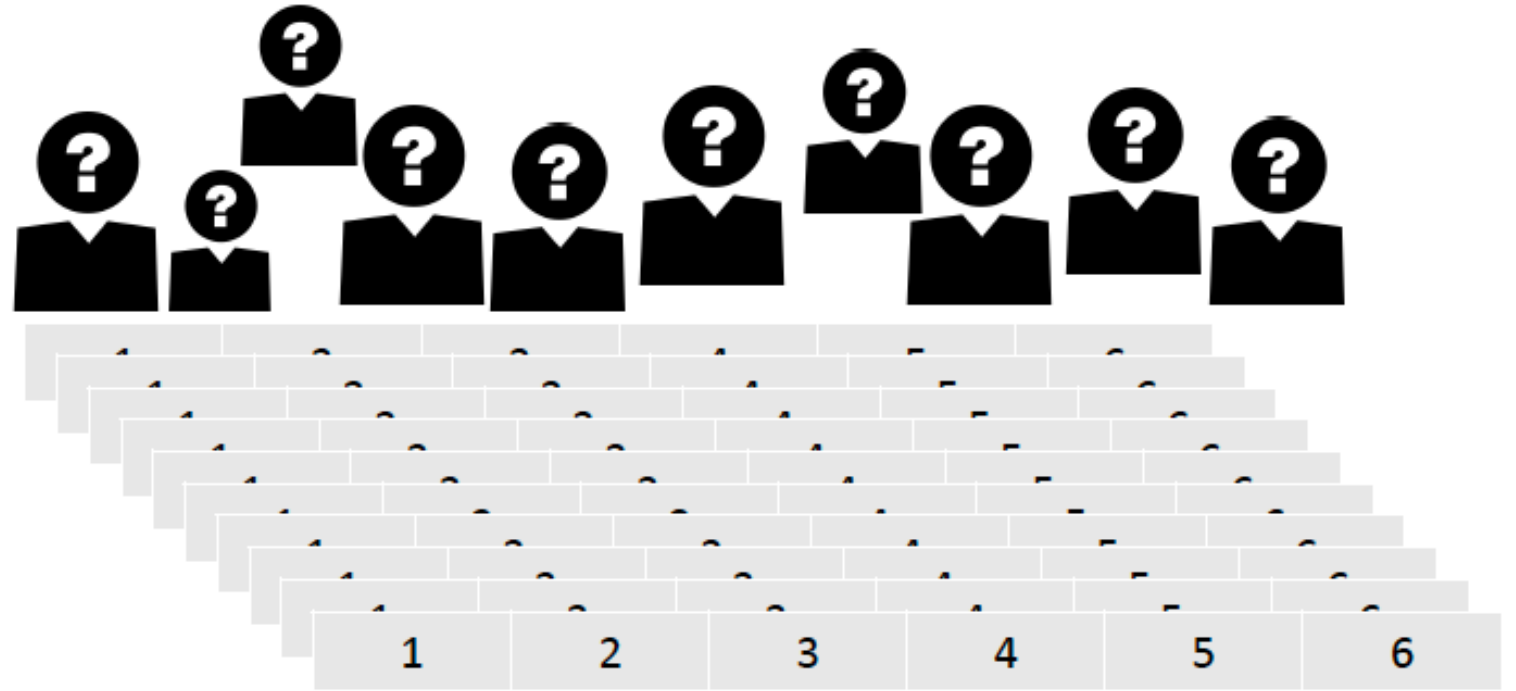
Phase1: Initial population

Generate population

- Define number of population
- Randomly create each individual, that represent the state of problem

e.g.;

Population = 10



	Sukhothai	Phitsanulok	Phetchabun	Kamphaeng Phet	Phichit	Nakhon Sawan
Sukhothai		125	226	82	144	185
Phitsanulok	125		147	178	127	202
Phetchabun	226	147		214	107	183
Kamphaeng Phet	82	178	214		118	121
Phichit	144	127	107	118		87
Nakhon Sawan	185	202	183	121	87	

Phase2: Selection for breeding offsprings

Fitness : find the shortest path → can be create on your own

example: Method1: the minus of the path's distance

Chromosome1:

1	2	3	4	5	6
---	---	---	---	---	---

Path1

$$\text{Distance} = D(1,2) + D(2,3) + D(3,4) + D(4,5) + D(5,6) + D(6,1) = 125 + 147 + 214 + 118 + 87 + 185 = 875$$

$$\text{fitness value} = -875$$

Chromosome2:

2	4	6	1	3	5
---	---	---	---	---	---

Path2

$$\text{Distance} = D(2,4) + D(4,6) + D(6,1) + D(1,3) + D(3,5) + D(5,2) = 178 + 121 + 185 + 226 + 107 + 127 = 944$$

$$\text{fitness value} = -944$$

other method: ratio of distance: $(x - a)/y$

let a, b, c, d, e = length of path

$$x = a + b + c + d + e$$

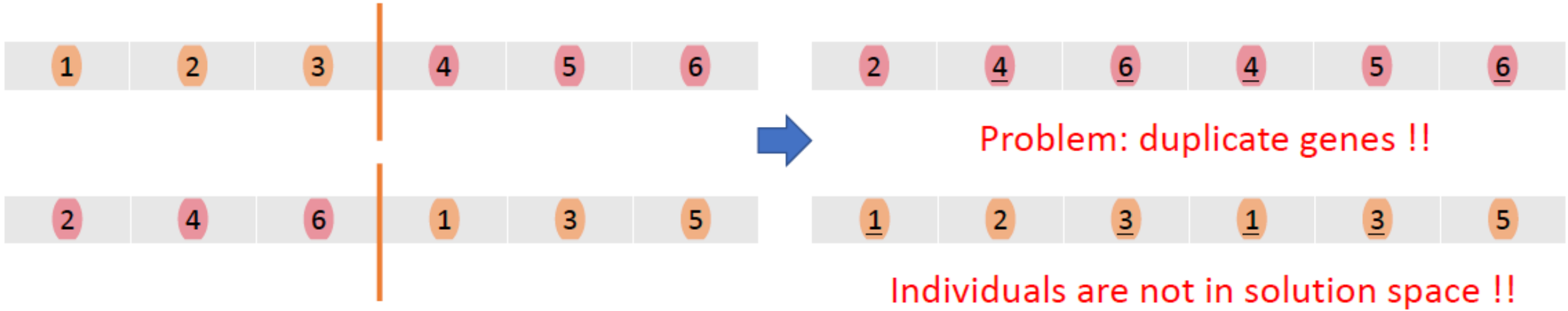
$$y = (x - a) + (x - b) + (x - c) + (x - d) + (x - e)$$

or

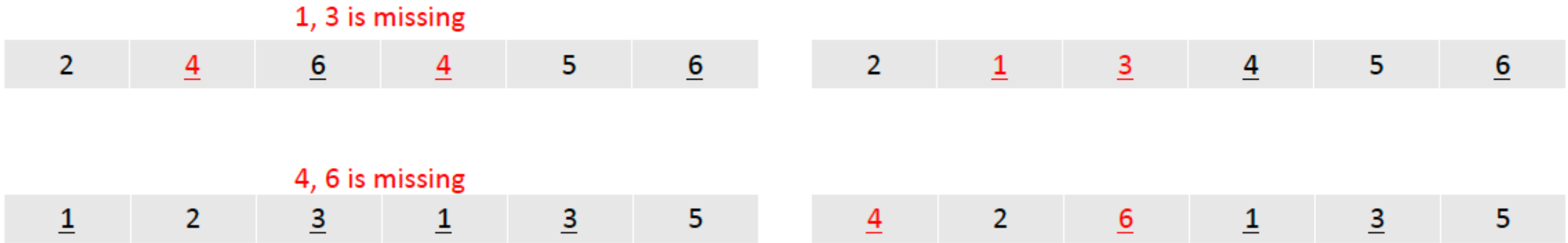
$$x = a^2 + b^2 + c^2 + d^2 + e^2$$

$$y = (x - a^2) + (x - b^2) + (x - c^2) + (x - d^2) + (x - e^2)$$

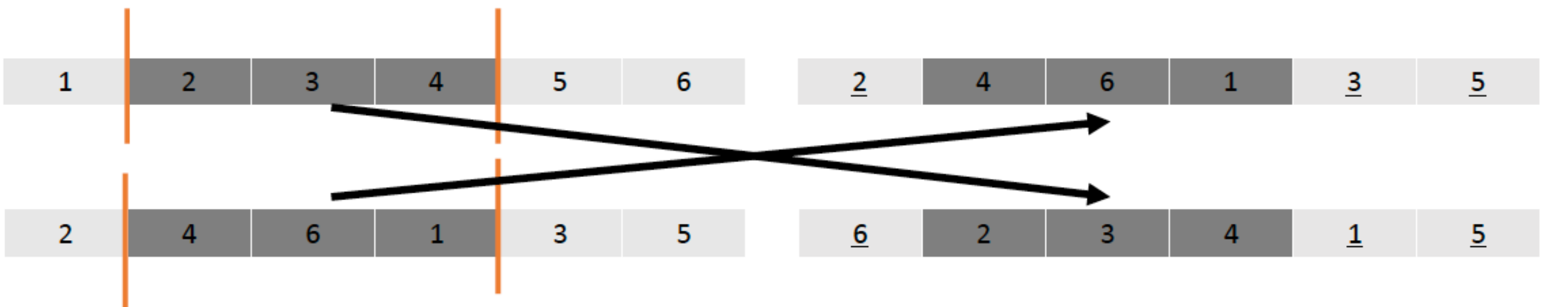
Crossover Problem



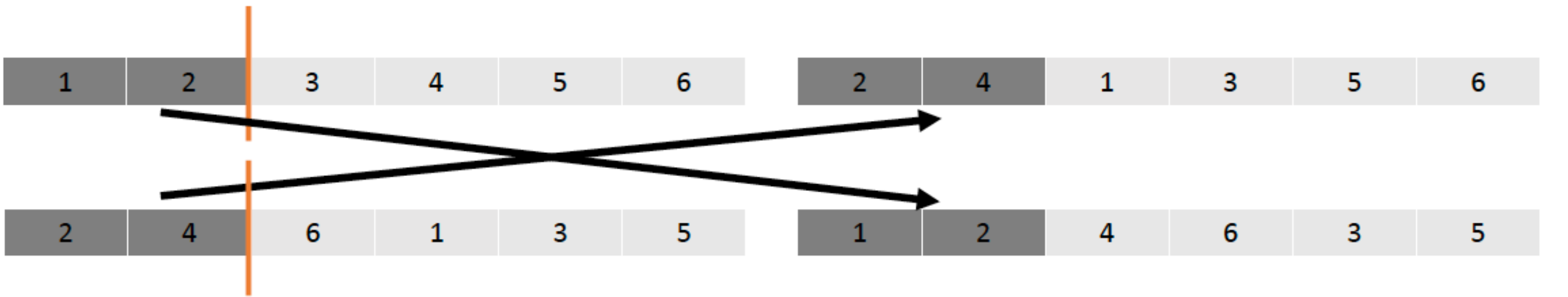
Fix method1: Directly change the duplicate genes with the missing genes



Fix method2: Recombine random part of 1st parent with the rest genes from 2nd parent in order



Fix method 3: Random single point crossover and copy the other rest genes from 2nd parent in order



Fix method4: Change chromosome representation (Recommended): Fix order position

position 1	position 2	position 3	position 4	position 5	position 6
Sukhothai	Phitsanulok	Phetchabun	Kamphaeng Phet	Phichit	Nakhon Sawan

1. Random number to represent chromosomes e.g.: random [0..1000]

chromosome1	700	150	400	200	300	100
chromosome2	500	450	210	130	250	120



New chromosome1	700	150	400	130	250	120
New chromosome2	500	450	210	200	300	100

2. To fitness: sorting the position in order

position 1	position 2	position 3	position 4	position 5	position 6
Sukhothai	Phitsanulok	Phetchabun	Kamphaeng Phet	Phichit	Nakhon Sawan

		sorted order					
		1	4	2	5	3	6
New chromosome1		700	150	400	130	250	120
Decoding		Sukhothai	Kamphaeng Phet	Phitsanulok	Phichit	Phetchabun	Nakhon Sawan

		sorted order					
		1	2	4	5	3	6
New chromosome2		500	450	210	200	300	100
Decoding		Sukhothai	Phitsanulok	Kamphaeng Phet	Phichit	Phetchabun	Nakhon Sawan

Then the chromosomes have no duplicated genes

Mutation method

method1: Randomly swap position



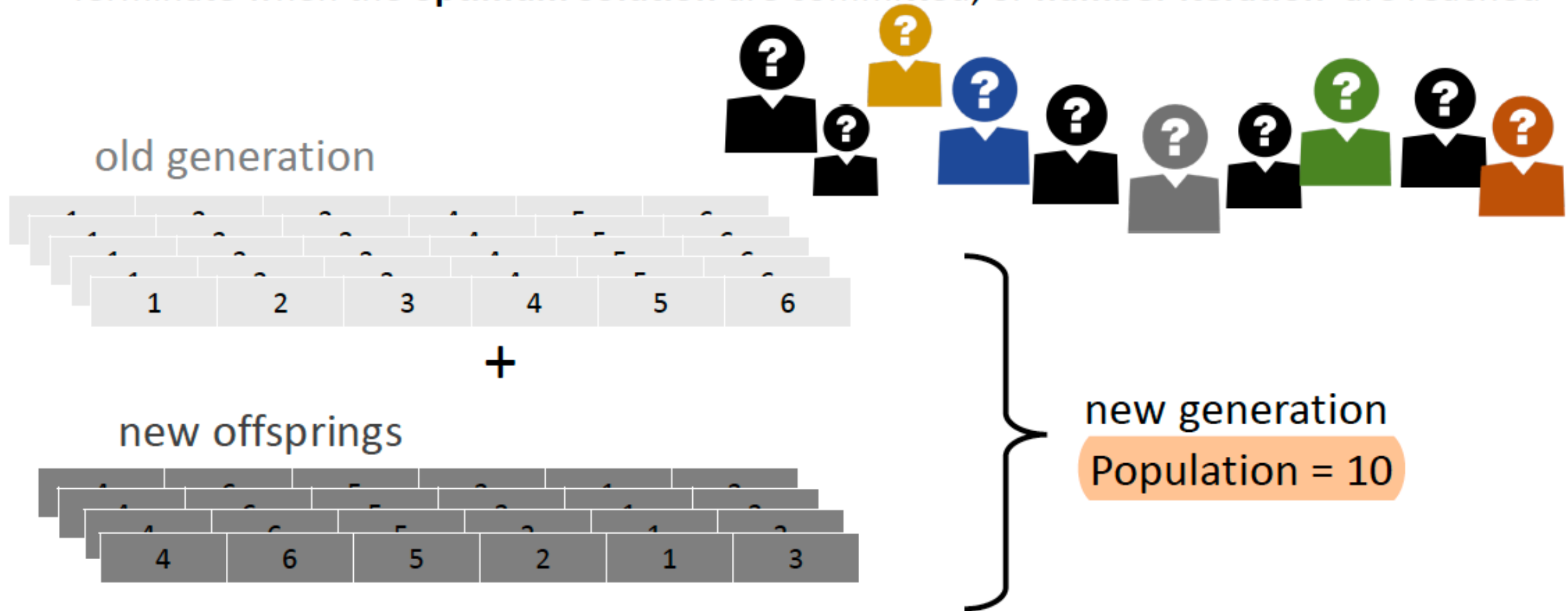
method2: Random pick and shift position



method3:

Phase3: Evaluation of a population

- Values of the fitness are computed for individuals of **the new offspring population**
- **Replace the old generation** with the higher value fitness offsprings
- Terminate when the **optimum solution** are committed, or **number iteration** are reached



Summary

- GA is one of the most popular biology-inspired approach
- GA use fitness function as heuristic function in heuristic search
- GA is used to avoid getting stuck in local maximum
- GA is flexible and there are many way to customize each processes e.g.;
Chromosome representation, fitness, reproduction
- GA dose not guarantee the best solution



Mini Project:

Problem: Travel Planning

City	Days to stay	Cost
City A	3	\$80
City B	4	\$85
City C	5	\$100
City D	2	\$45
City E	3	\$75
City F	1	\$25
City G	6	\$148
City H	8	\$205
City I	1	\$24
City J	7	\$200

Assume that there are long vacations for 14 days.
Try to plan a trip to visit these cities with these criteria:

- use minimum cost
- travel many cities as many as we can
- stay within 14 days

Compare and discuss these 5 plans
from using brute-force, dynamic programming,
greedy technique and GA

+ 1 plan from other Bio-inspired Algorithms in paper